

Architecture Document

Project Title: *Shipment Pricing Prediction*

Domain: *Supply Chain Analytics*

Difficulty Level: *Intermediate*

Cloud Provider: *AWS*

Table of Contents

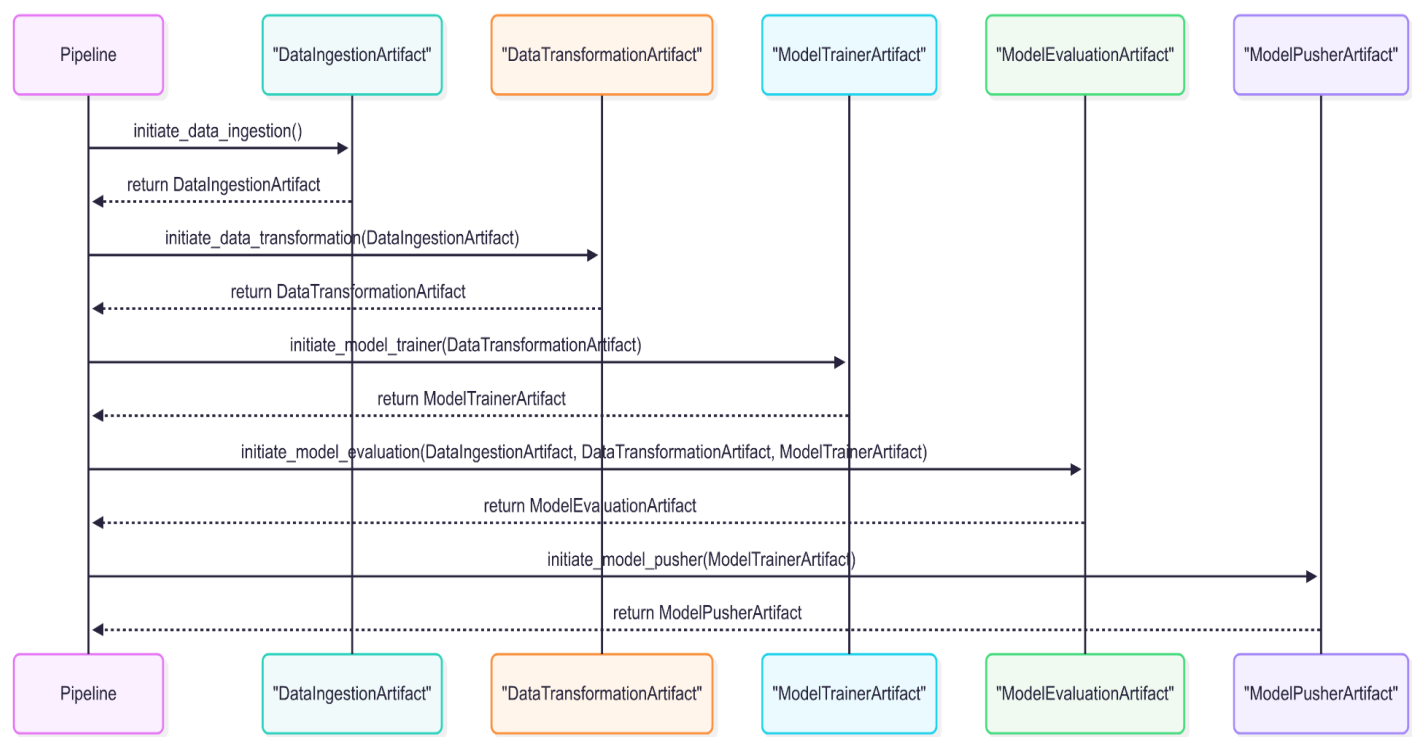
1. Introduction
 2. High-Level Architecture
 3. Component-Level Architecture (Flowcharts, Explanations, and Sequence Diagrams)
 - 3.1 Data Ingestion
 - 3.2 Data Validation
 - 3.3 Data Transformation
 - 3.4 Model Trainer
 - 3.5 Model Evaluation
 - 3.6 Model Pusher
 - 3.7 Cost Model
 - 3.8 S3 to Local
 4. Pipeline-Level Architecture (Flowcharts, Explanations, and Sequence Diagrams)
 - 4.1 Training Pipeline
 - 4.2 Prediction Pipeline
 5. Application Layer (FastAPI)
 6. Deployment & Infrastructure
 7. Error Handling & Logging
 8. Security & Compliance
 9. Conclusion
-

1. Introduction

The Shipment Price Prediction system is designed to automate data ingestion, validation, transformation, model training, evaluation, and deployment for predicting shipment prices. This document provides a detailed architectural view of the system, including **flowcharts**, **sequence diagrams**, and **process explanations**.

2. High-Level System Architecture

System Execution Flow:



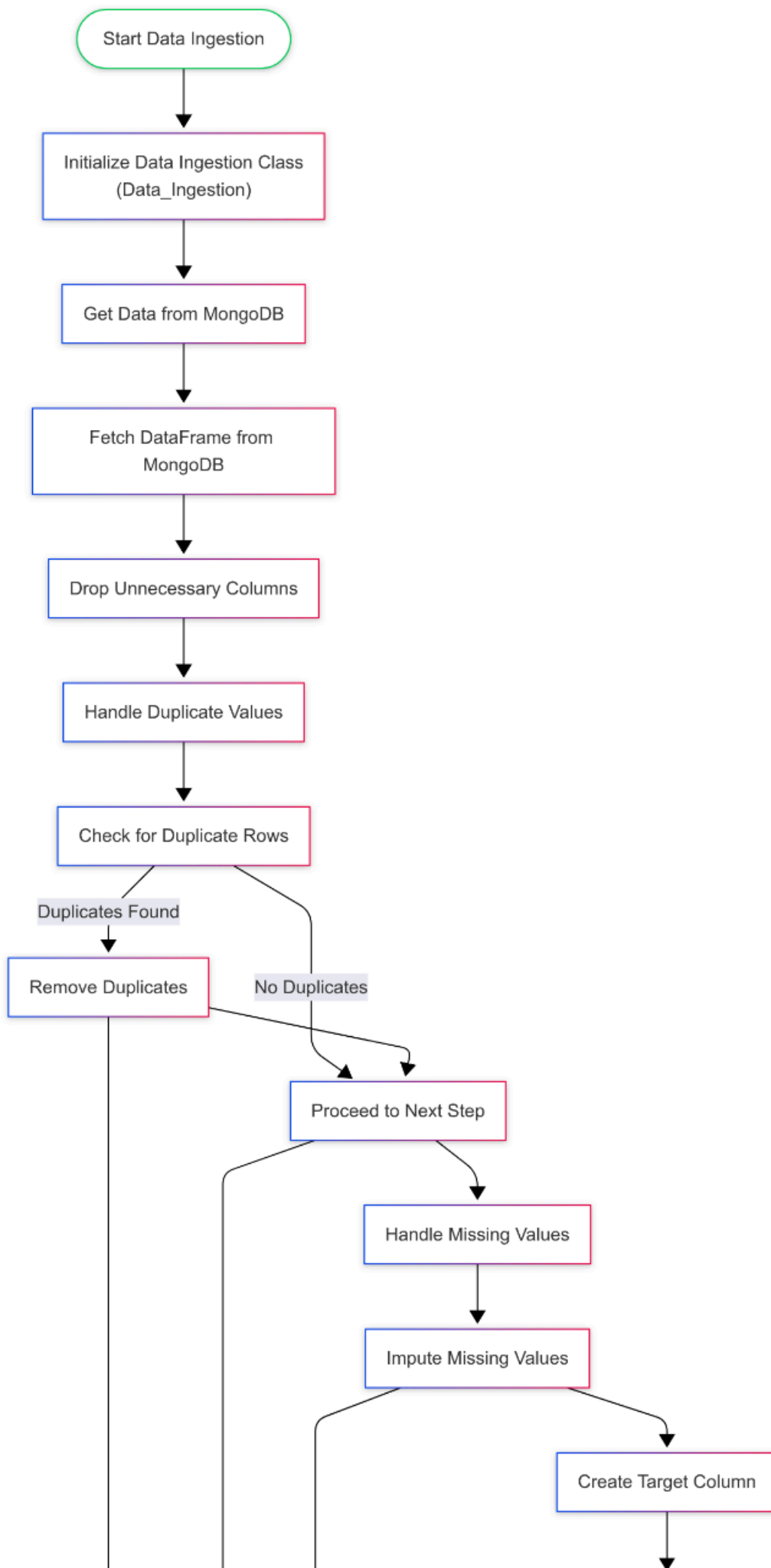
3. Component-Level Architecture

Each component is explained with its **flowchart, sequence diagram, and flow explanation**.

3.1 Data Ingestion

Description: Responsible for fetching raw shipment data from MongoDB, cleaning it, handling duplicates/missing values, and splitting into train/test sets.

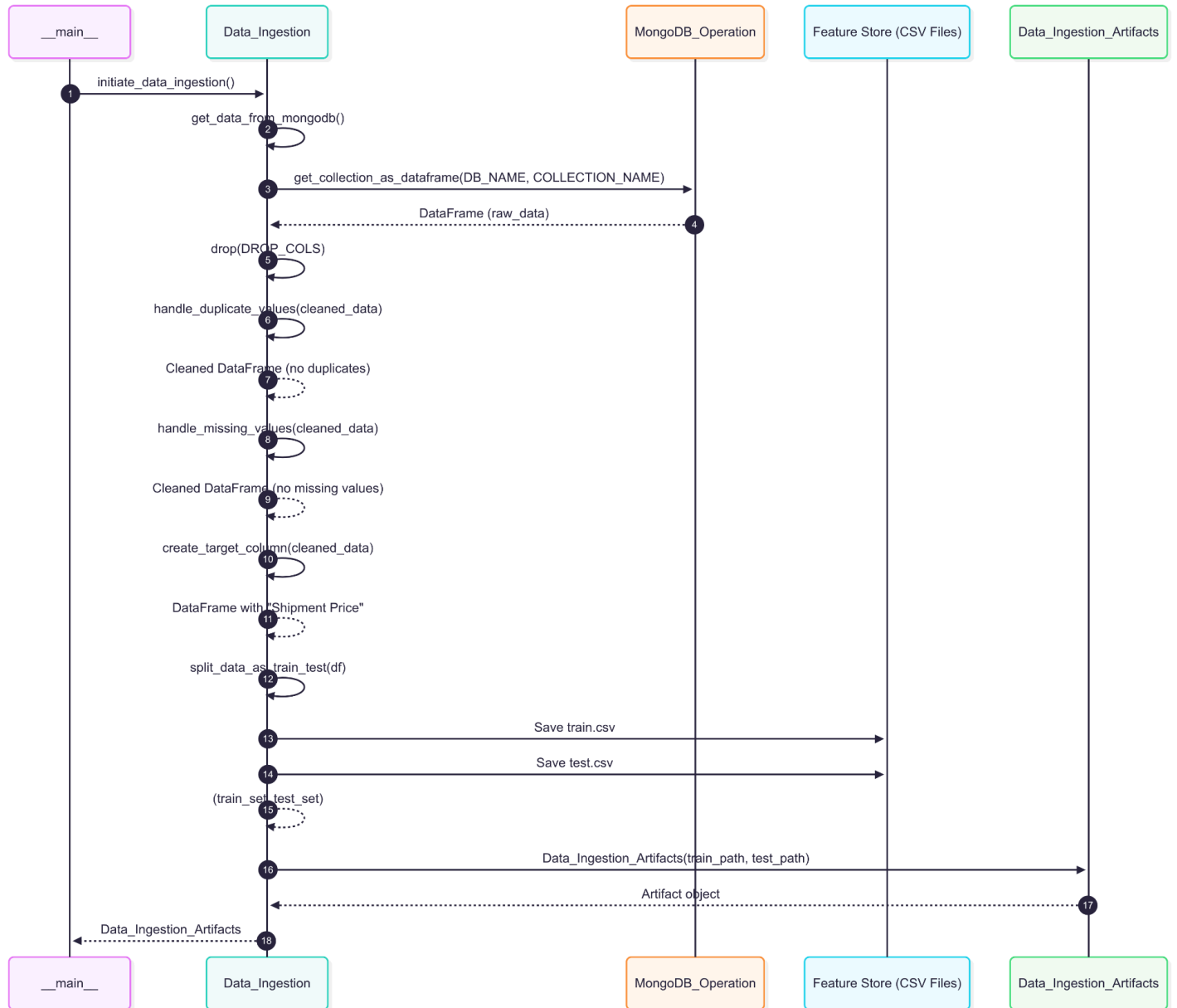
Flowchart Image here:



Explanation of Flowchart Components Data_Ingestion:

- **Start Data Ingestion:** The entry point of the data ingestion process.
- **Initialize Data Ingestion Class:** Instantiation of the Data_Ingestion class.
- **Get Data from MongoDB:** Fetching data from the MongoDB database.
- **Fetch DataFrame from MongoDB:** Retrieving the data as a DataFrame.
- **Drop Unnecessary Columns:** Removing columns that are not needed for analysis.
- **Handle Duplicate Values:** Checking and removing duplicate rows if found.
- **Check for Duplicate Rows:** Identifying duplicates in the dataset.
- **Remove Duplicates:** Removing duplicate entries from the dataset.
- **Proceed to Next Step:** Continuing to the next step if no duplicates are found.
- **Handle Missing Values:** Identifying and handling missing values in the dataset.
- **Impute Missing Values:** Applying imputation strategies to fill in missing values.
- **Create Target Column:** Creating the target variable for the dataset.
- **Construct 'Shipment Price' Target Column:** Summing specific columns to create the target variable.
- **Split Data into Train and Test Sets:** Splitting the cleaned dataset into training and testing datasets.
- **Load Cleaned Dataset:** Loading the cleaned dataset for splitting.
- **Split Data Based on Train-Test Ratio:** Performing the split based on the specified ratio.
- **Save Train Data as train.csv:** Saving the training dataset to a CSV file.
- **Save Test Data as test.csv:** Saving the testing dataset to a CSV file.
- **Data Ingestion Completed Successfully:** Indicating the successful completion of the data ingestion process.
- **End Data Ingestion:** The exit point of the data ingestion process.
- **Log Error & Stop Pipeline:** Handling any errors that occur during the ingestion process.

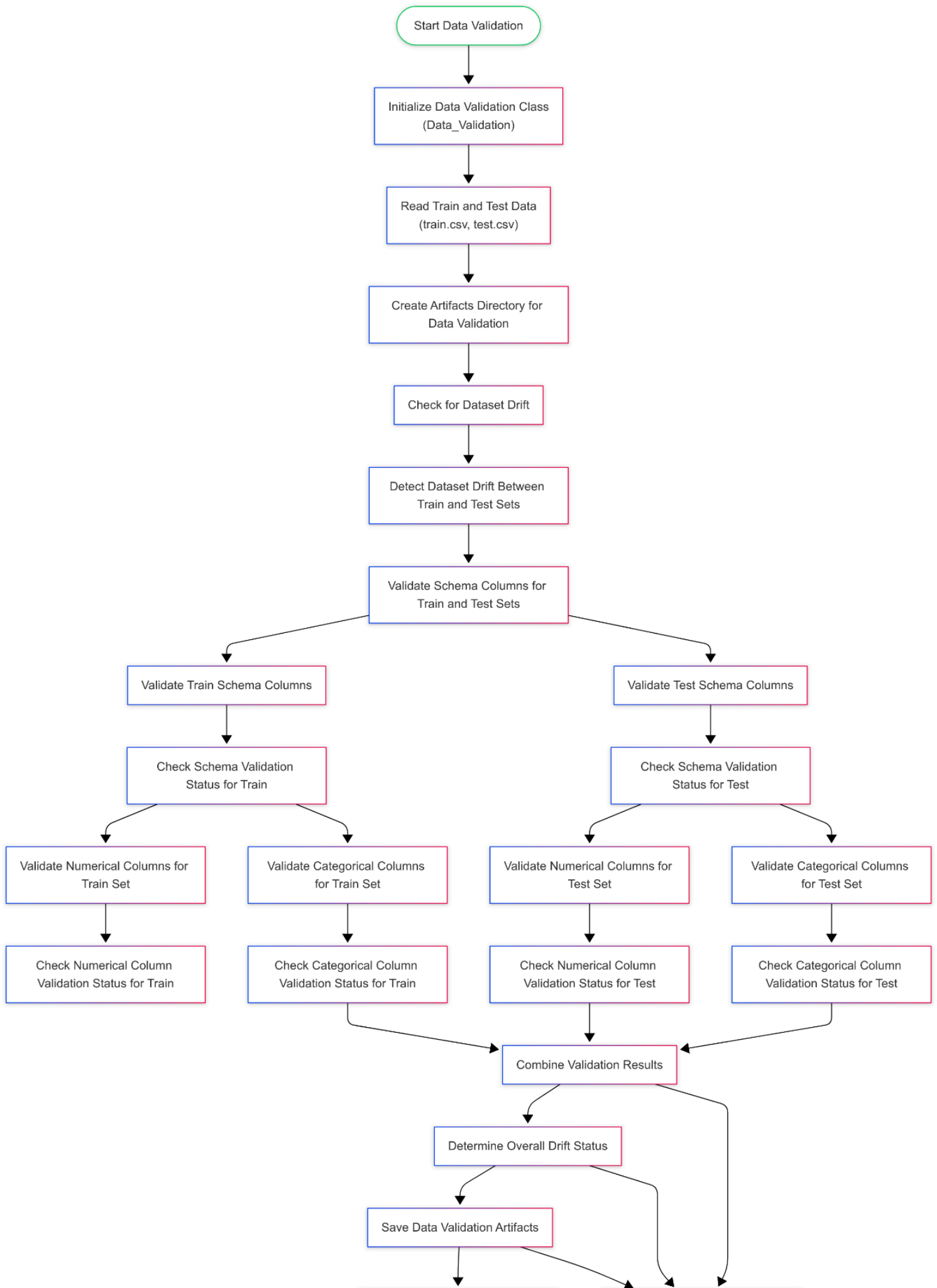
Sequence Diagram Image here:



3.2 Data Validation

Description: Ensures the ingested data matches schema, checks for drift, and validates categorical/numerical columns.

Flowchart Image here:

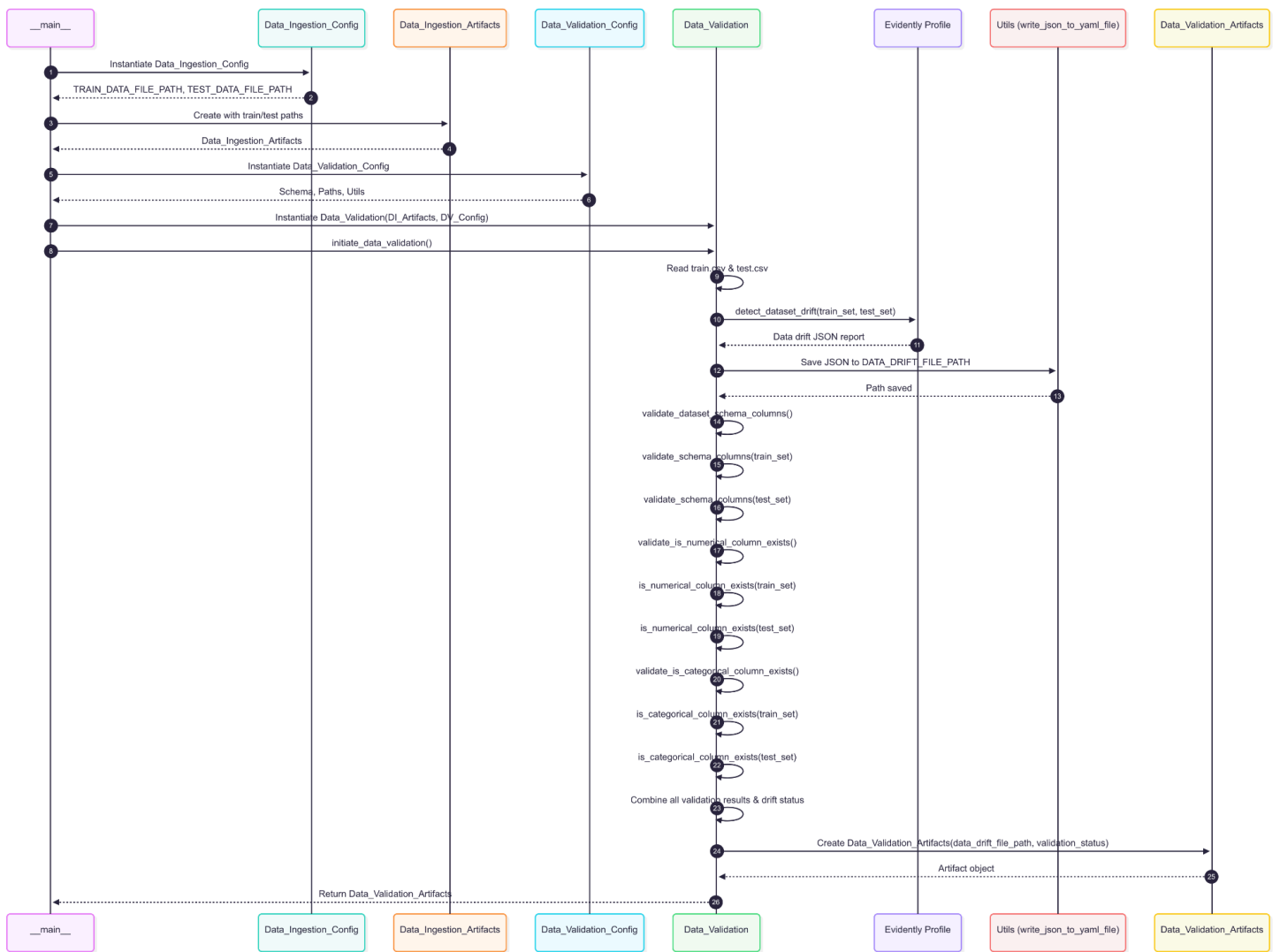


Explanation of Flowchart Components Data_Validation:

- **Start Data Validation:** The entry point of the data validation process.
- **Initialize Data Validation Class:** Instantiation of the Data_Validation class.
- **Read Train and Test Data:** Loading the datasets from specified file paths.
- **Create Artifacts Directory for Data Validation:** Creating a directory to save validation artifacts.
- **Check for Dataset Drift:** Detecting drift between the training and testing datasets.
- **Detect Dataset Drift Between Train and Test Sets:** Analyzing drift using the `detect_dataset_drift` method.
- **Validate Schema Columns for Train and Test Sets:** Validating the schema of both datasets.
- **Validate Train Schema Columns:** Checking if the schema columns in the training dataset match the expected schema.
- **Validate Test Schema Columns:** Checking if the schema columns in the testing dataset match the expected schema.
- **Check Schema Validation Status for Train:** Assessing the validation status of the training dataset.
- **Check Schema Validation Status for Test:** Assessing the validation status of the testing dataset.
- **Validate Numerical Columns for Train Set:** Checking for the existence of numerical columns in the training dataset.
- **Check Numerical Column Validation Status for Train:** Assessing the validation status of numerical columns in the training dataset.
- **Validate Numerical Columns for Test Set:** Checking for the existence of numerical columns in the testing dataset.
- **Check Numerical Column Validation Status for Test:** Assessing the validation status of numerical columns in the testing dataset.
- **Validate Categorical Columns for Train Set:** Checking for the existence of categorical columns in the training dataset.
- **Check Categorical Column Validation Status for Train:** Assessing the validation status of categorical columns in the training dataset.

- **Validate Categorical Columns for Test Set:** Checking for the existence of categorical columns in the testing dataset.
- **Check Categorical Column Validation Status for Test:** Assessing the validation status of categorical columns in the testing dataset.
- **Combine Validation Results:** Aggregating the results of all validation checks.
- **Determine Overall Drift Status:** Assessing whether the overall validation was successful or not.
- **Save Data Validation Artifacts:** Saving the results and reports of the validation process.
- **Data Validation Completed Successfully:** Indicating the successful completion of the data validation process.
- **End Data Validation:** The exit point of the data validation process.
- **Handle Validation Error & Stop Pipeline:** Handling any errors that occur during the validation process.

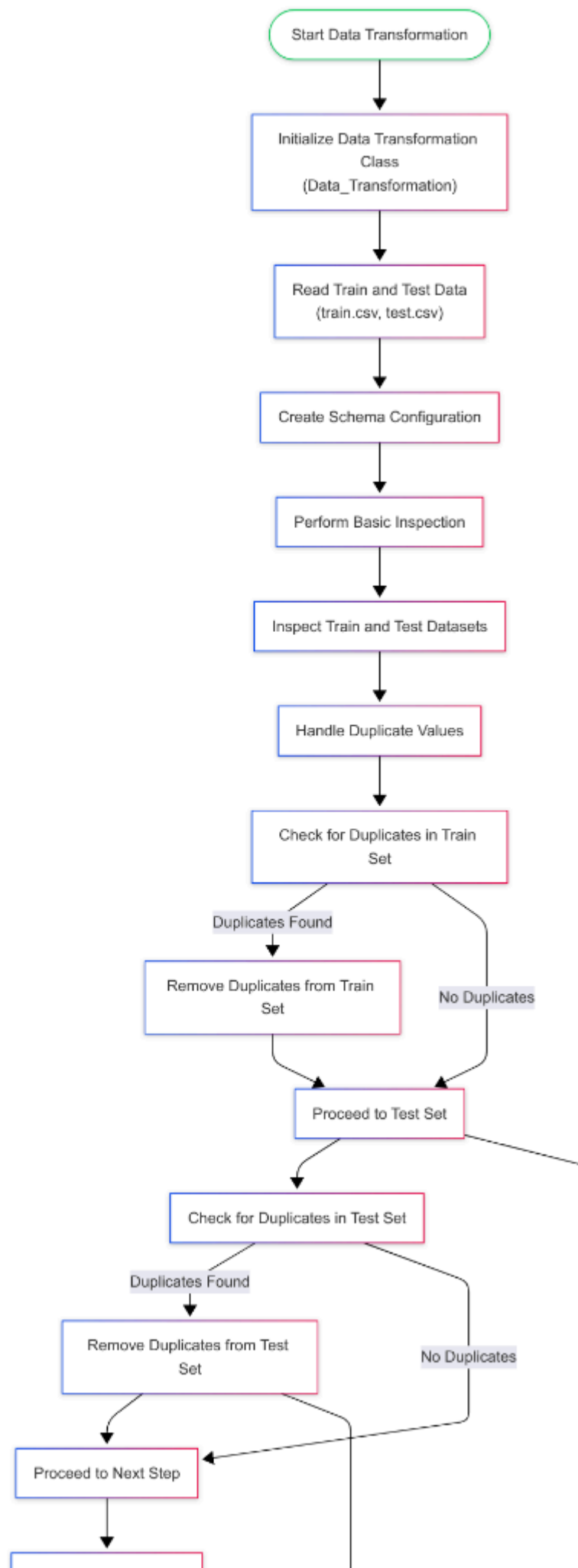
Sequence Diagram Image here:



3.3 Data Transformation

Description: Prepares datasets for ML training, including imputation, encoding, outlier handling, and preprocessing pipelines.

Flowchart Image here:

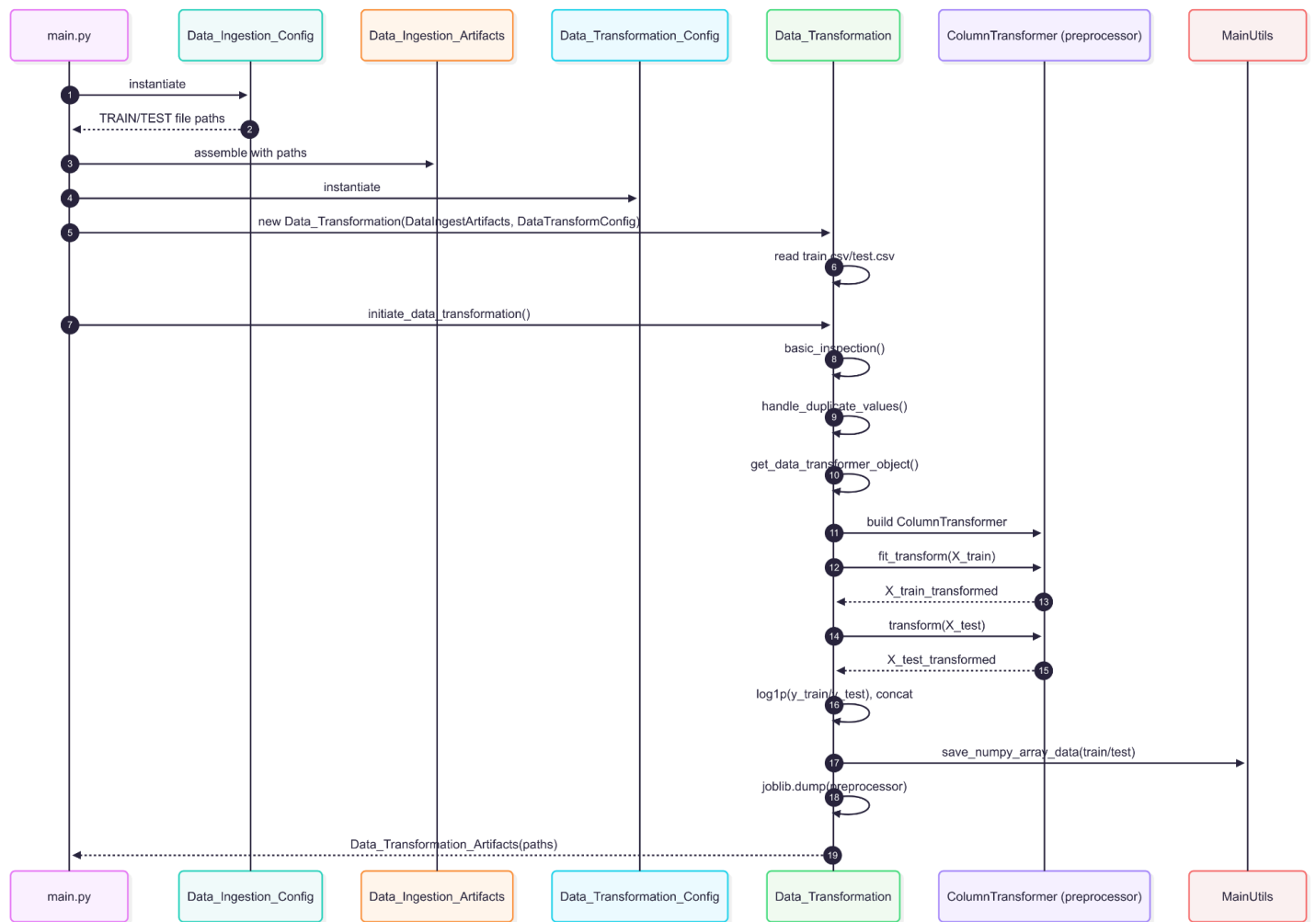


Explanation of Flowchart Components Data_Transformation :

- **Start Data Transformation:** The entry point of the data transformation process.
- **Initialize Data Transformation Class:** Instantiation of the Data_Transformation class.
- **Read Train and Test Data:** Loading the datasets from specified file paths.
- **Create Schema Configuration:** Reading the schema configuration for data transformation.
- **Perform Basic Inspection:** Conducting an initial inspection of the datasets.
- **Inspect Train and Test Datasets:** Analyzing the structure and quality of both datasets.
- **Handle Duplicate Values:** Checking and removing duplicate rows from the datasets.
- **Check for Duplicates in Train Set:** Identifying duplicates in the training dataset.
- **Remove Duplicates from Train Set:** Removing duplicate entries from the training dataset.
- **Proceed to Test Set:** Continuing to check for duplicates in the testing dataset.
- **Check for Duplicates in Test Set:** Identifying duplicates in the testing dataset.
- **Remove Duplicates from Test Set:** Removing duplicate entries from the testing dataset.
- **Proceed to Next Step:** Continuing to the next step if no duplicates are found.
- **Handle Missing Values:** Identifying and handling missing values in the datasets.
- **Impute Missing Values for Numerical Columns:** Applying imputation strategies for numerical columns.
- **Impute Missing Values for Categorical Columns:** Applying imputation strategies for categorical columns.
- **Handle Outliers:** Managing outliers in the datasets.
- **Apply Winsorization to Numerical Columns:** Capping extreme values in numerical columns.
- **Create Target Column:** Creating the target variable for the dataset.

- **Construct 'Shipment Price' Target Column:** Summing specific columns to create the target variable.
- **Get Data Transformer Object:** Retrieving the preprocessor object for data transformation.
- **Create Preprocessor Pipeline:** Setting up the transformation pipeline.
- **Fit and Transform Training Data:** Applying the preprocessor to the training dataset.
- **Transform Test Data:** Applying the preprocessor to the testing dataset.
- **Save Transformed Data as Artifacts:** Saving the processed datasets and preprocessor object for future use.
- **Data Transformation Completed Successfully:** Indicating the successful completion of the data transformation process.
- **End Data Transformation:** The exit point of the data transformation process.
- **Handle Validation Error & Stop Pipeline:** Handling any errors that occur during the transformation process.

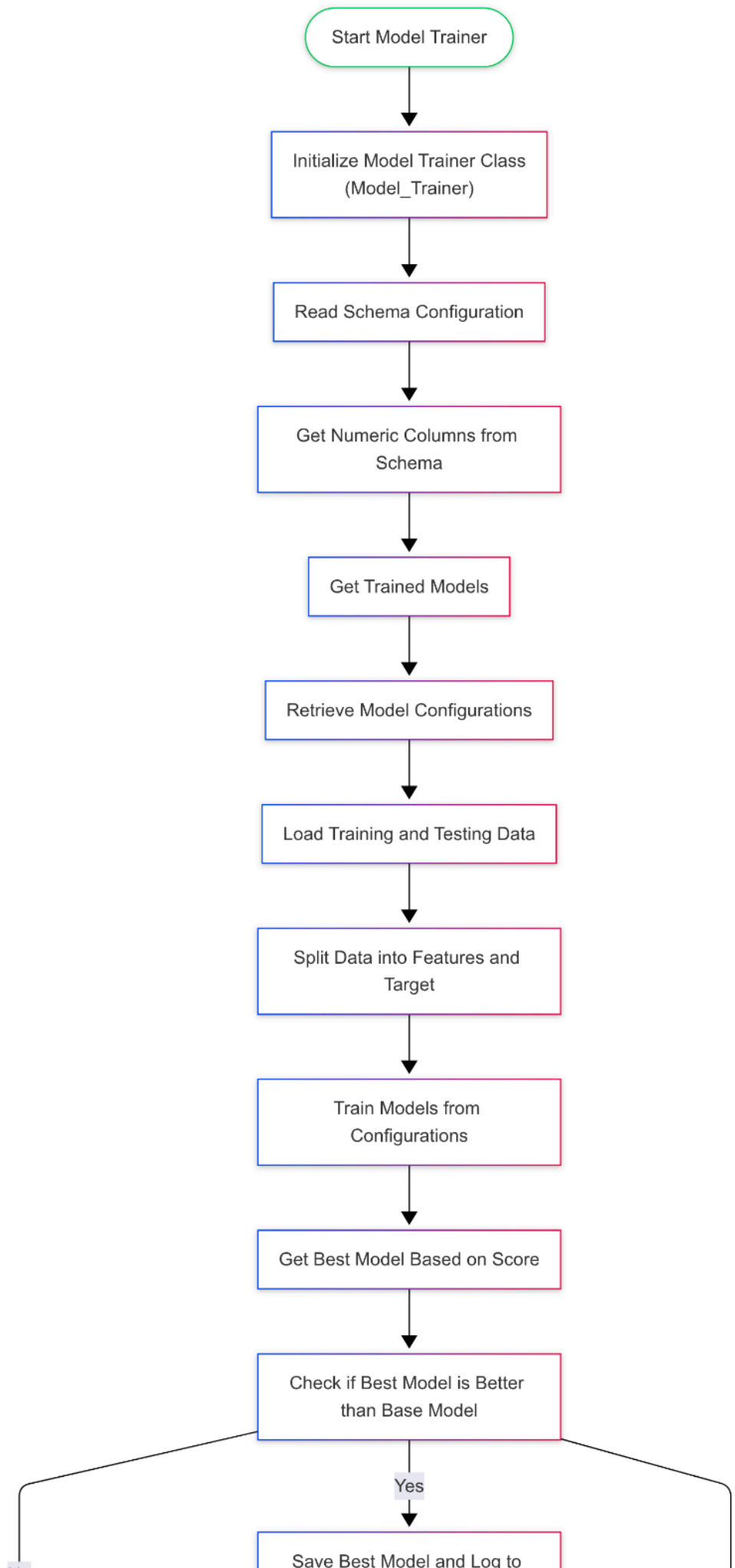
Sequence Diagram Image here:



3.4 Model Trainer

Description: Trains multiple ML models, selects the best model, and saves it.

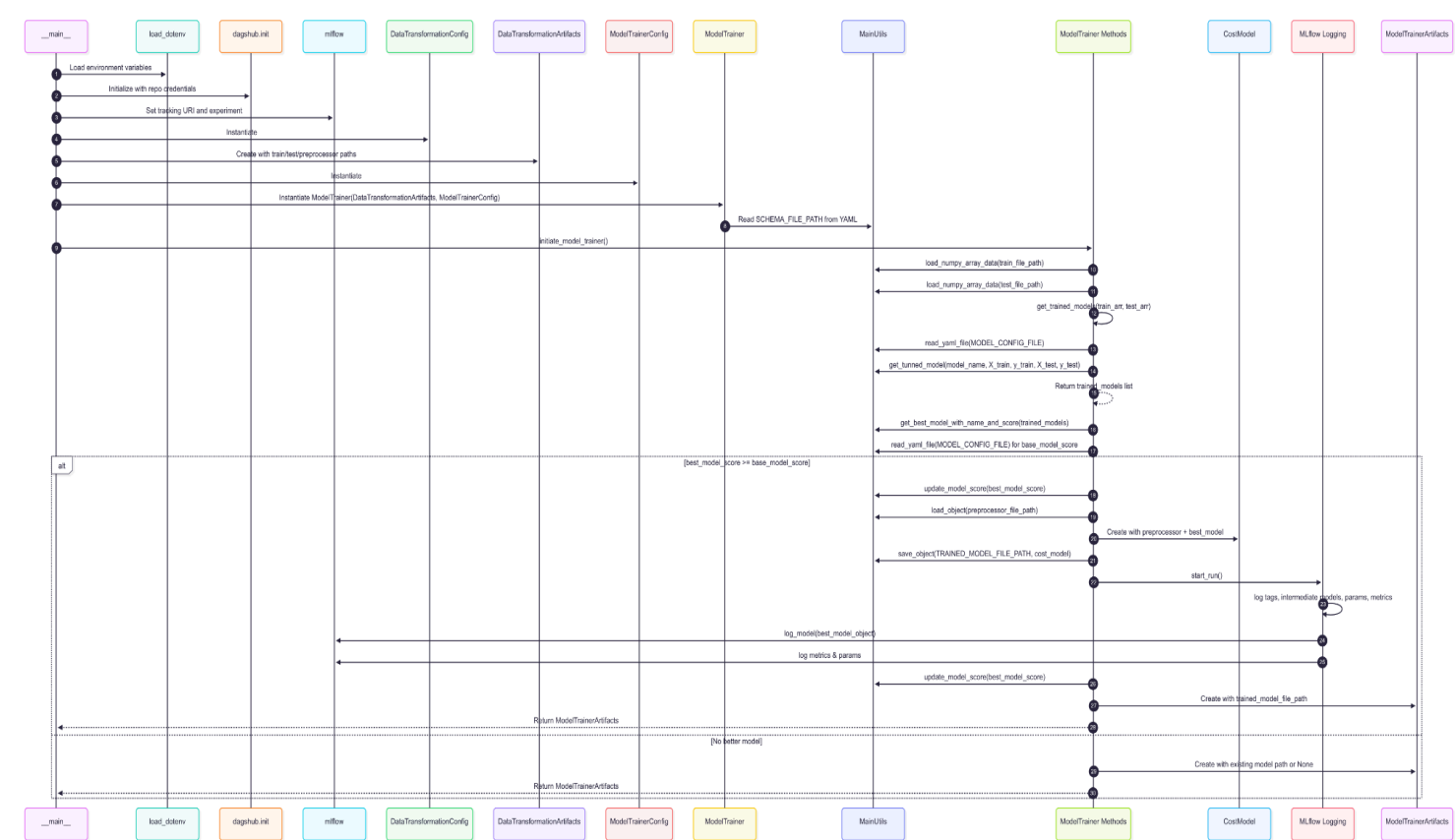
Flowchart Image here:



Explanation of Flowchart Components `Model_Trainer`:

- **Start Model Trainer:** The entry point of the model training process.
- **Initialize Model Trainer Class:** Instantiation of the `Model_Trainer` class.
- **Read Schema Configuration:** Loading the schema configuration for model training.
- **Get Numeric Columns from Schema:** Retrieving the numeric feature columns from the schema.
- **Get Trained Models:** Initiating the process to retrieve and train models.
- **Retrieve Model Configurations:** Loading model configurations from the specified file.
- **Load Training and Testing Data:** Reading the training and testing datasets.
- **Split Data into Features and Target:** Separating the features and target variable from the datasets.
- **Train Models from Configurations:** Training the models based on the configurations provided.
- **Get Best Model Based on Score:** Identifying the best model based on performance metrics.
- **Check if Best Model is Better than Base Model:** Comparing the best model's score with the base model's score.
- **Save Best Model and Log to MLflow:** Saving the best model and logging relevant information to MLflow.
- **Save Model Artifacts:** Storing the model artifacts for future use.
- **Model Training Completed Successfully:** Indicating the successful completion of the model training process.
- **Use Existing Model:** If no better model is found, using the existing model.
- **Return Existing Model Artifacts:** Returning the artifacts of the existing model.
- **End Model Trainer:** The exit point of the model training process.
- **Handle Validation Error & Stop Pipeline:** Handling any errors that occur during the training process.

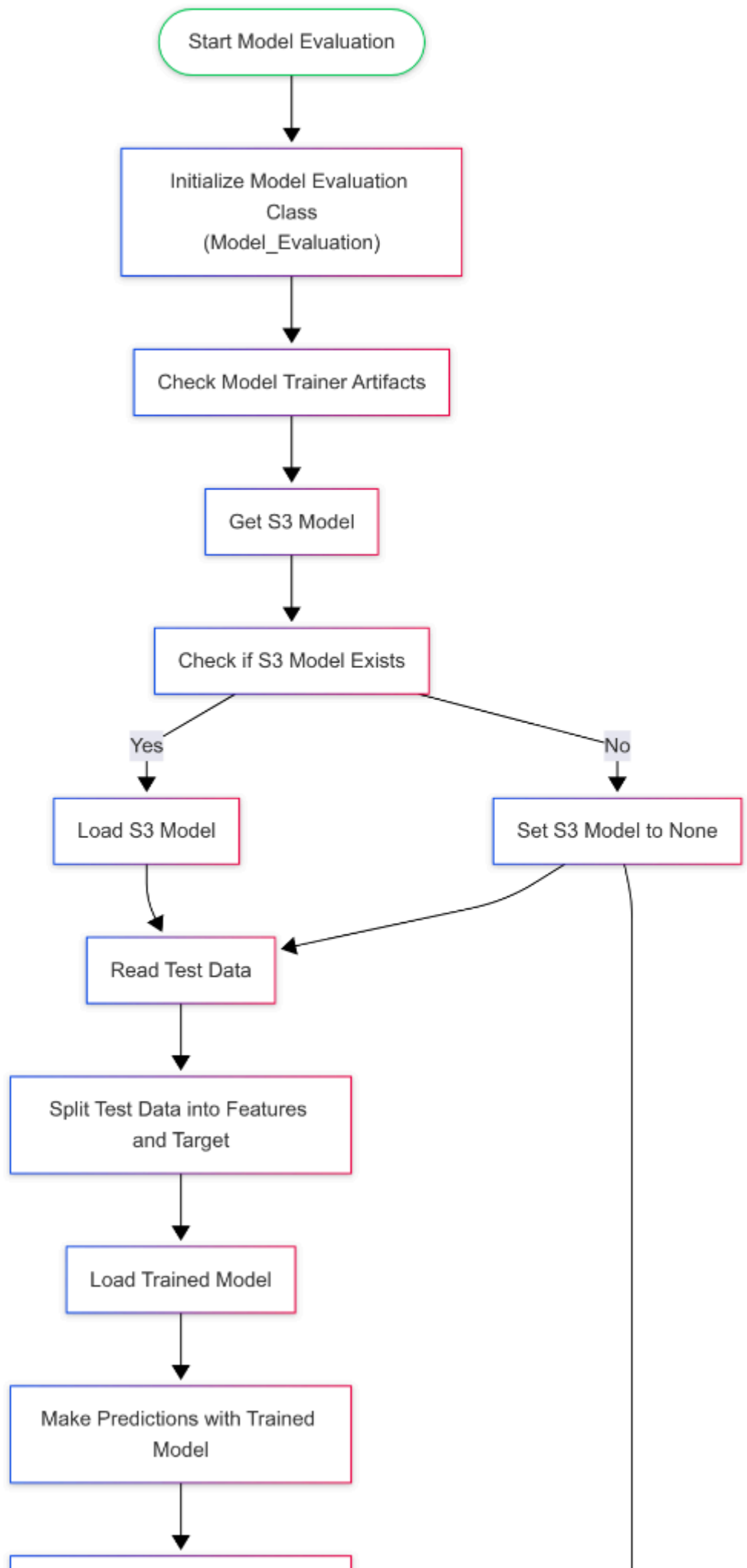
Sequence Diagram Image here:



3.5 Model Evaluation

Description: Compares the new trained model with the best S3-stored model using R² score and selects the better one.

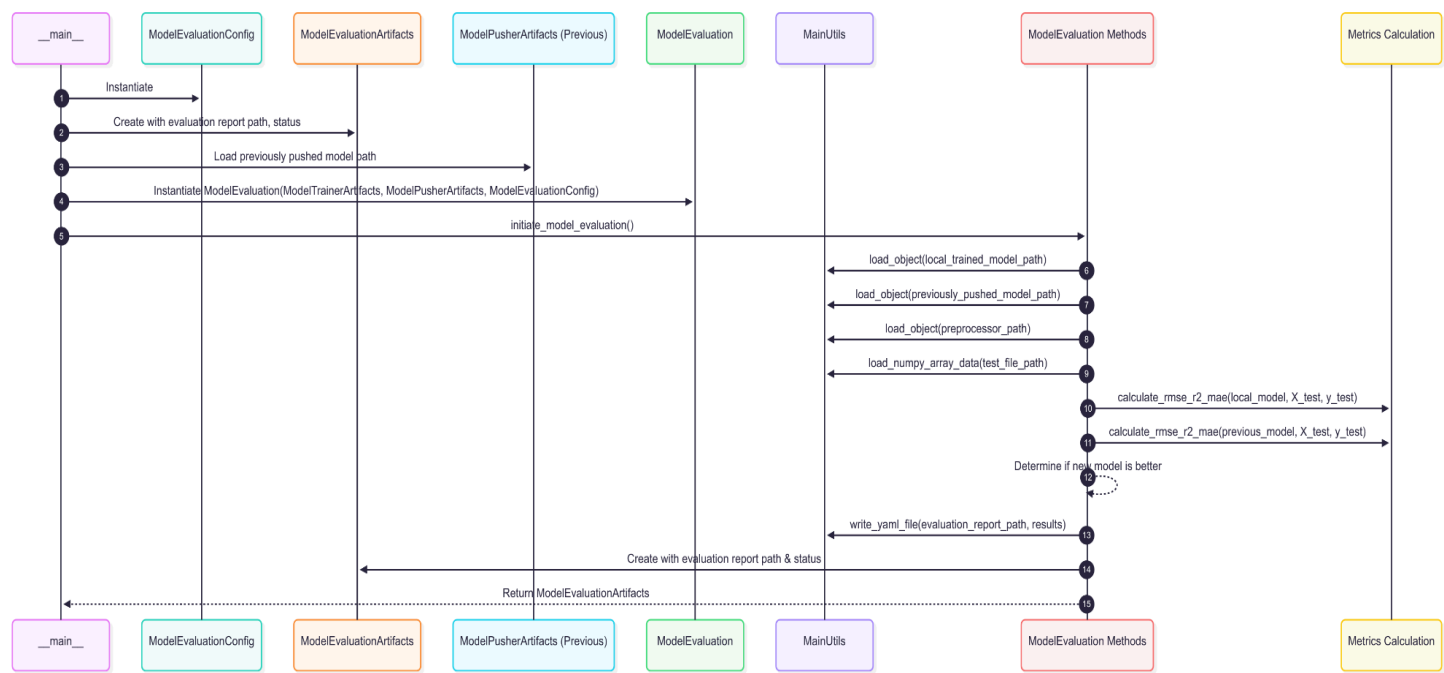
Flowchart Image here:



Explanation of Flowchart Components `Model_Trainer`:

- **Start Model Trainer:** The entry point of the model training process.
- **Initialize Model Trainer Class:** Instantiation of the `Model_Trainer` class.
- **Read Schema Configuration:** Loading the schema configuration for model training.
- **Get Numeric Columns from Schema:** Retrieving the numeric feature columns from the schema.
- **Get Trained Models:** Initiating the process to retrieve and train models.
- **Retrieve Model Configurations:** Loading model configurations from the specified file.
- **Load Training and Testing Data:** Reading the training and testing datasets.
- **Split Data into Features and Target:** Separating the features and target variable from the datasets.
- **Train Models from Configurations:** Training the models based on the configurations provided.
- **Get Best Model Based on Score:** Identifying the best model based on performance metrics.
- **Check if Best Model is Better than Base Model:** Comparing the best model's score with the base model's score.
- **Save Best Model and Log to MLflow:** Saving the best model and logging relevant information to MLflow.
- **Save Model Artifacts:** Storing the model artifacts for future use.
- **Model Training Completed Successfully:** Indicating the successful completion of the model training process.
- **Use Existing Model:** If no better model is found, using the existing model.
- **Return Existing Model Artifacts:** Returning the artifacts of the existing model.
- **End Model Trainer:** The exit point of the model training process.
- **Handle Validation Error & Stop Pipeline:** Handling any errors that occur during the training process.

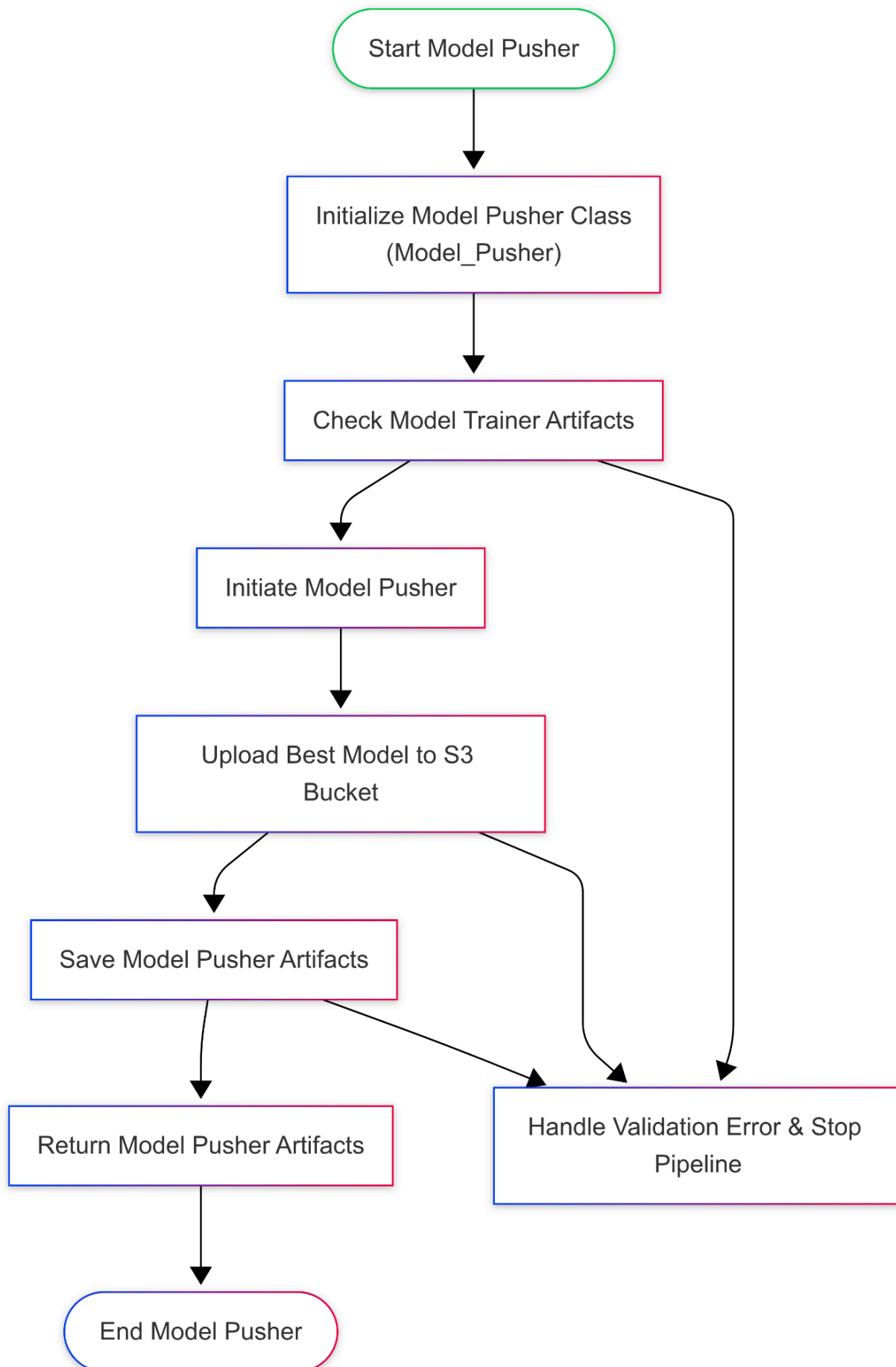
Sequence Diagram Image here:



3.6 Model Pusher

Description: Pushes the accepted best model to S3 for persistence and deployment.

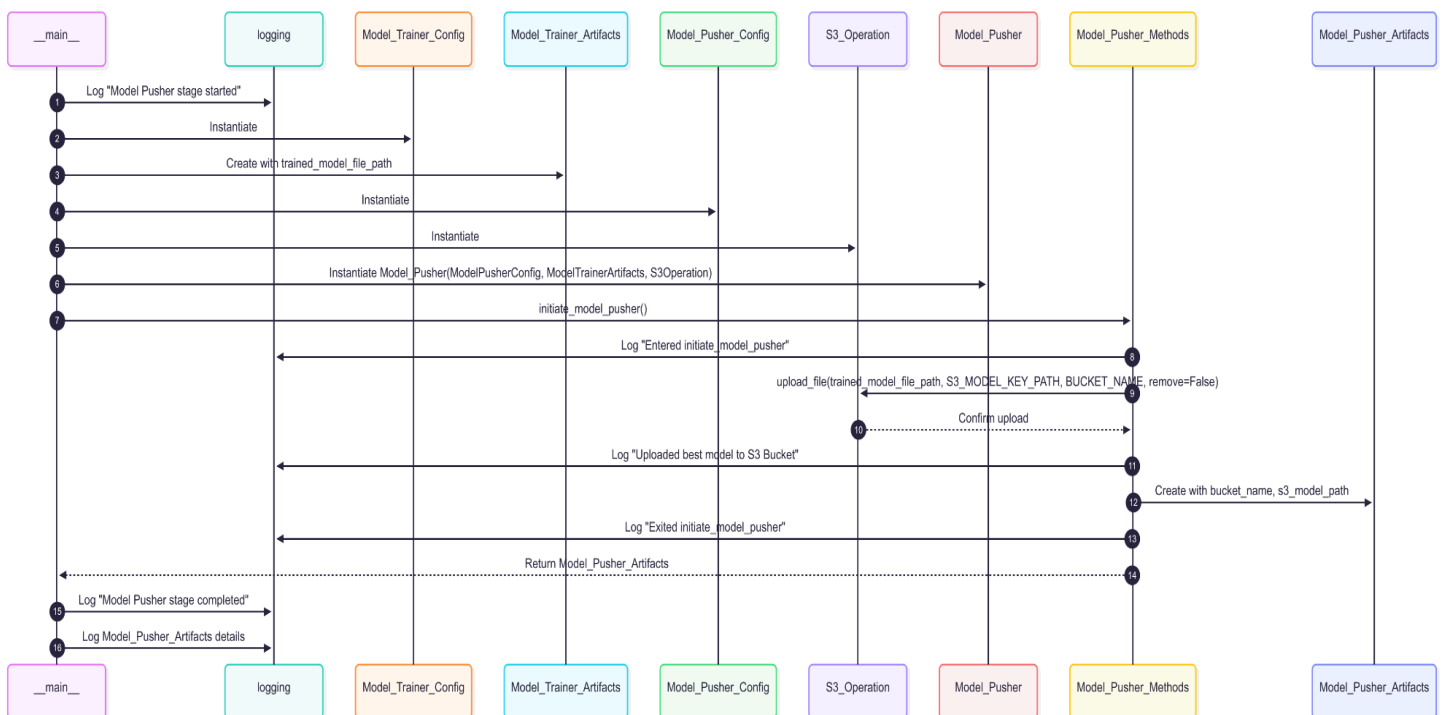
Flowchart Image here:



Explanation of Flowchart Components Model_Pusher :

- **Start Model Pusher:** The entry point of the model pusher process.
- **Initialize Model Pusher Class:** Instantiation of the Model_Pusher class.
- **Check Model Trainer Artifacts:** Verifying that the model trainer artifacts are properly initialized.
- **Initiate Model Pusher:** Starting the process to push the model to S3.
- **Upload Best Model to S3 Bucket:** Uploading the trained model to the specified S3 bucket.
- **Save Model Pusher Artifacts:** Saving the artifacts related to the model pusher.
- **Return Model Pusher Artifacts:** Returning the artifacts of the model pusher.
- **End Model Pusher:** The exit point of the model pusher process.
- **Handle Validation Error & Stop Pipeline:** Handling any errors that occur during the pusher process.

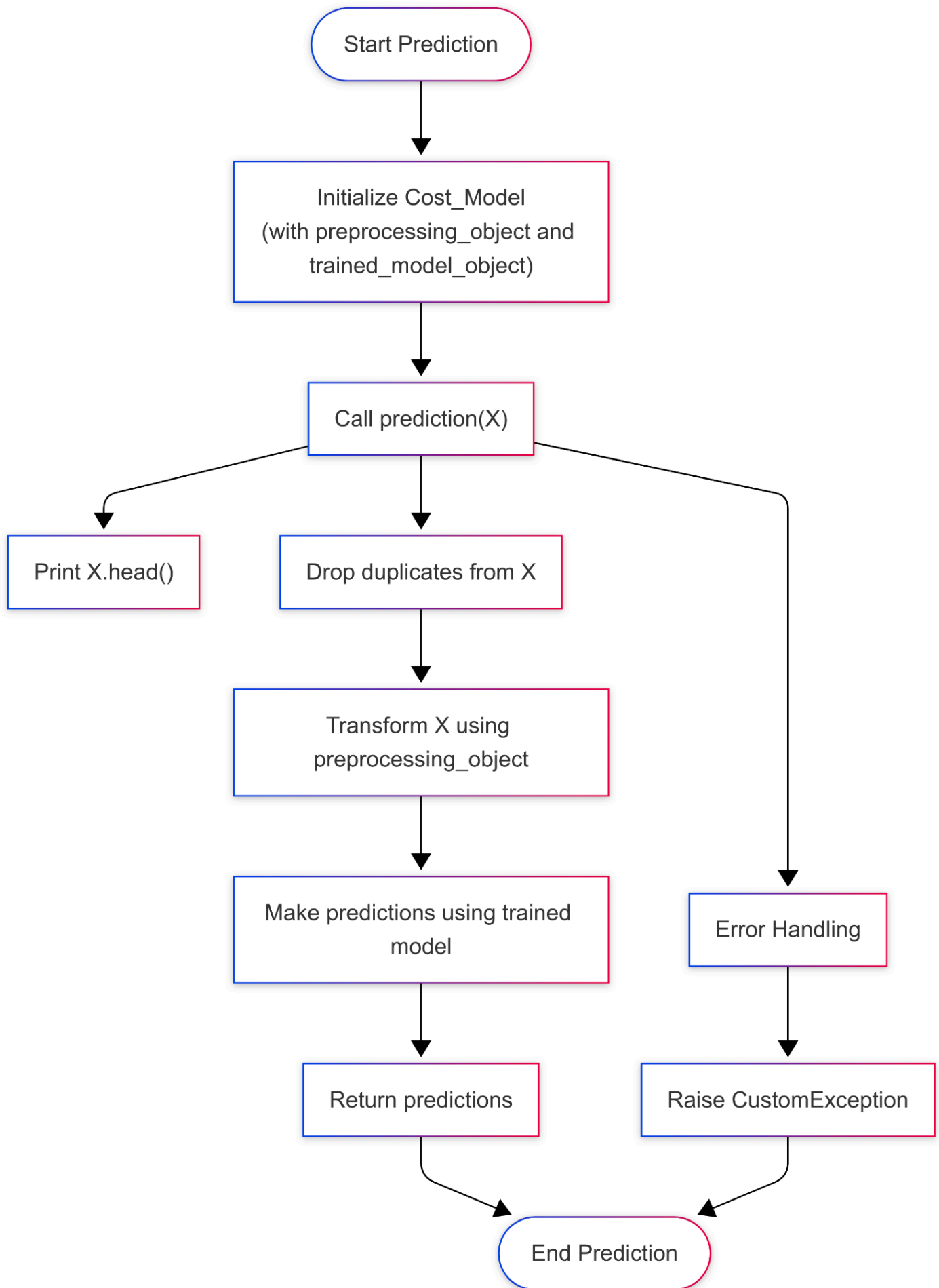
Sequence Diagram Image here:



3.7 Cost Model

Description: Uses the trained model to predict shipment price for incoming data.

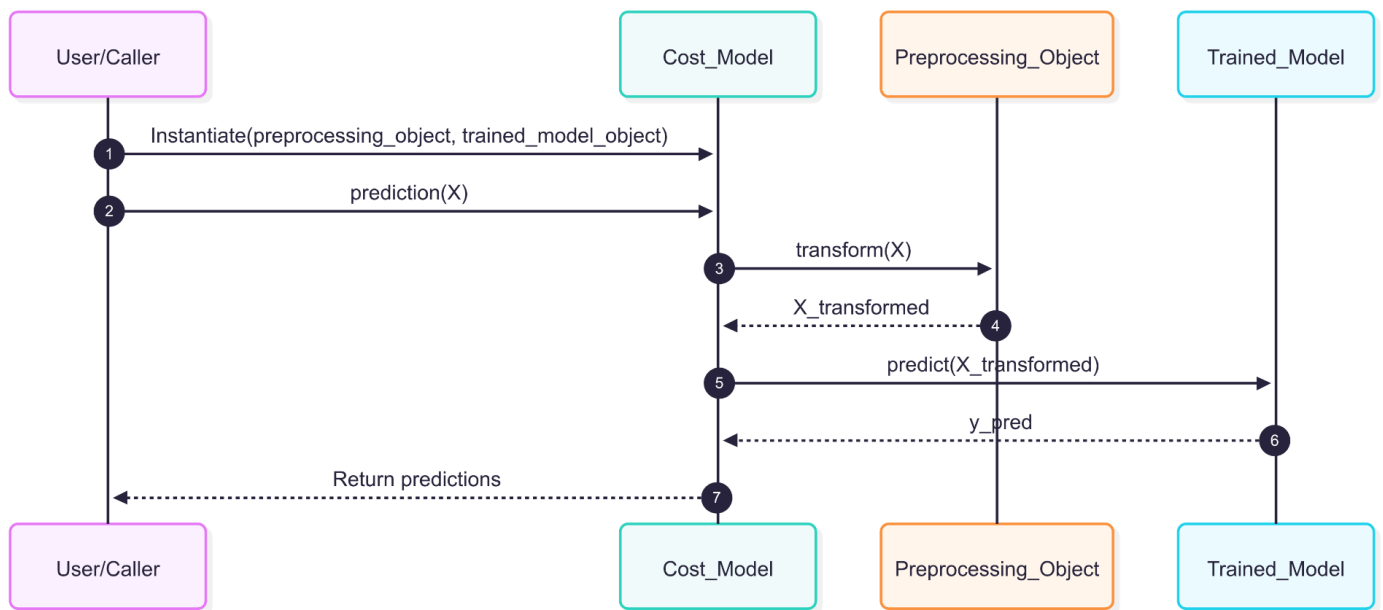
Flowchart Image here:



Explanation of Flowchart Components Cost_Model :

- **Start Prediction:** The entry point of the prediction process.
- **Initialize Cost_Model:** Instantiation of the Cost_Model class with the preprocessing and trained model objects.
- **Call prediction(X):** Invoking the prediction method with input data .
- **Print X.head():** Displaying the first few rows of the input data.
- **Drop duplicates from X:** Removing duplicate entries from the input data.
- **Transform X using preprocessing_object:** Applying the preprocessing steps to the input data.
- **Make predictions using the trained model:** Using the trained model to generate predictions based on the transformed data.
- **Return predictions:** Returning the predictions made by the model.
- **End Prediction:** The exit point of the prediction process.
- **Error Handling:** Handling any exceptions that occur during the prediction process.
- **Raise CustomException:** Raising a custom exception if an error occurs.

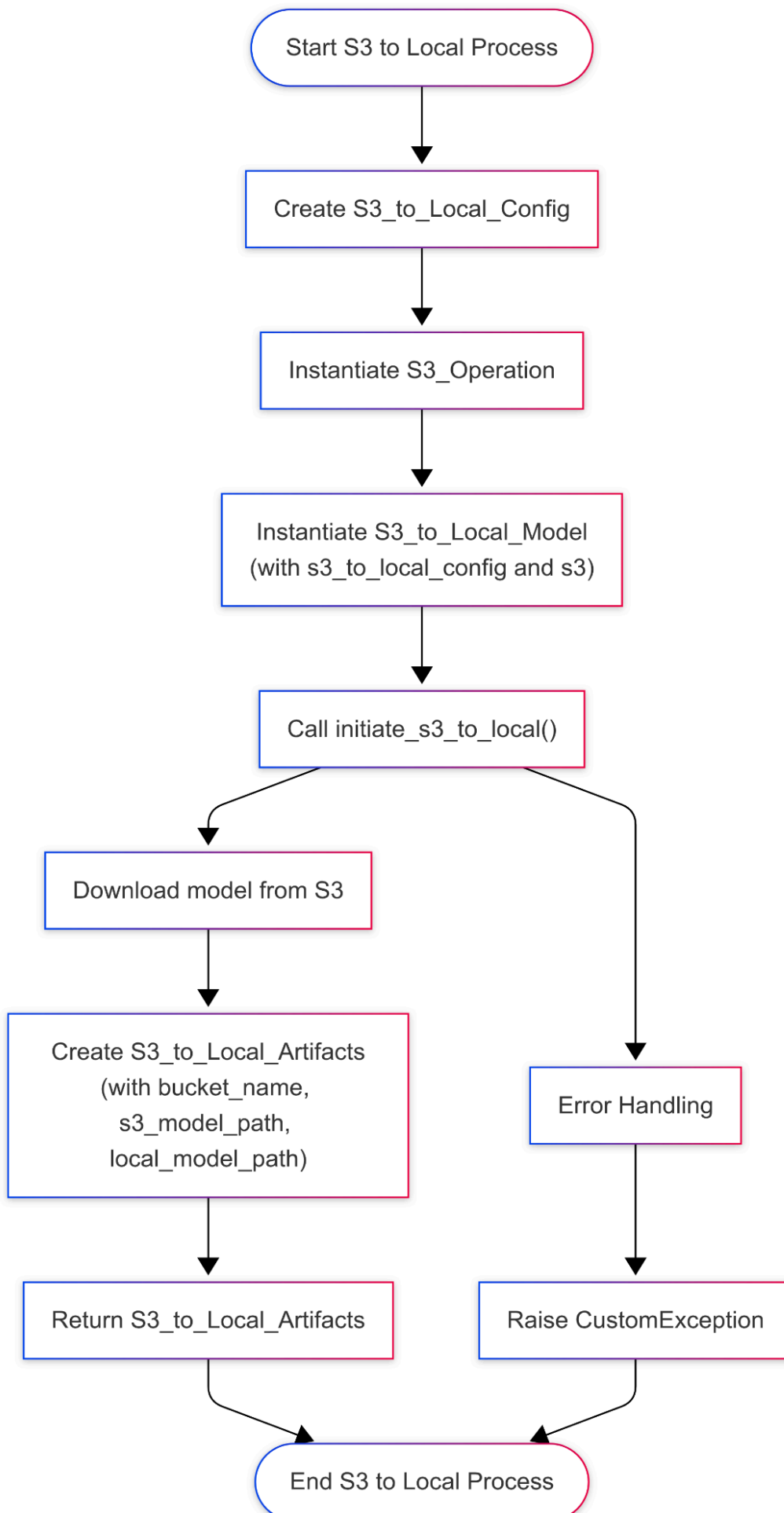
Sequence Diagram Image here:



3.8 S3 to Local

Description: Pulls best model artifacts from S3 to local for inference.

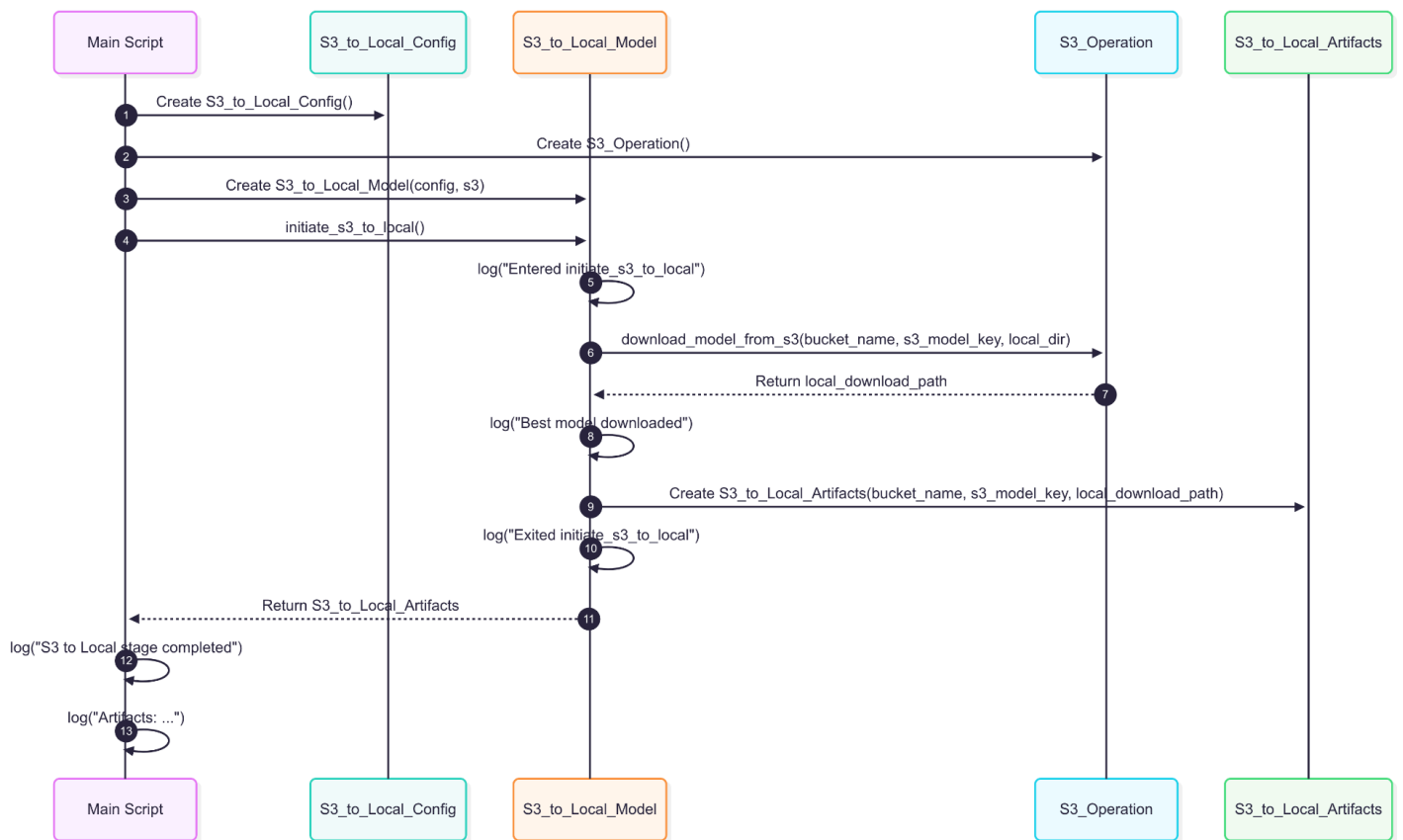
Flowchart Image here:



Explanation of Flowchart Components S3_to_Local:

- ***Start S3 to Local Process:*** The entry point of the S3 to local model process.
- ***Create S3_to_Local_Config:*** Creating the configuration object for S3 to local operations.
- ***Instantiate S3_Operation:*** Creating an instance of the S3_Operation class to handle S3 operations.
- ***Instantiate S3_to_Local_Model:*** Creating an instance of the S3_to_Local_Model class with the configuration and S3 operation objects.
- ***Call initiate_s3_to_local():*** Invoking the method to download the model from S3 to the local directory.
- ***Download model from S3:*** Executing the download operation from S3.
- ***Create S3_to_Local_Artifacts:*** Creating an artifact object that contains information about the downloaded model.
- ***Return S3_to_Local_Artifacts:*** Returning the artifact containing the model details.
- ***End S3 to Local Process:*** The exit point of the S3 to local model process.
- ***Error Handling:*** Handling any exceptions that occur during the process.
- ***Raise CustomException:*** Raising a custom exception if an error occurs.

Sequence Diagram Image here:



4. Pipeline-Level Architecture

4.1 Training Pipeline

Description: Orchestrates the end-to-end ML training process.

Flowchart of Training Pipeline here:



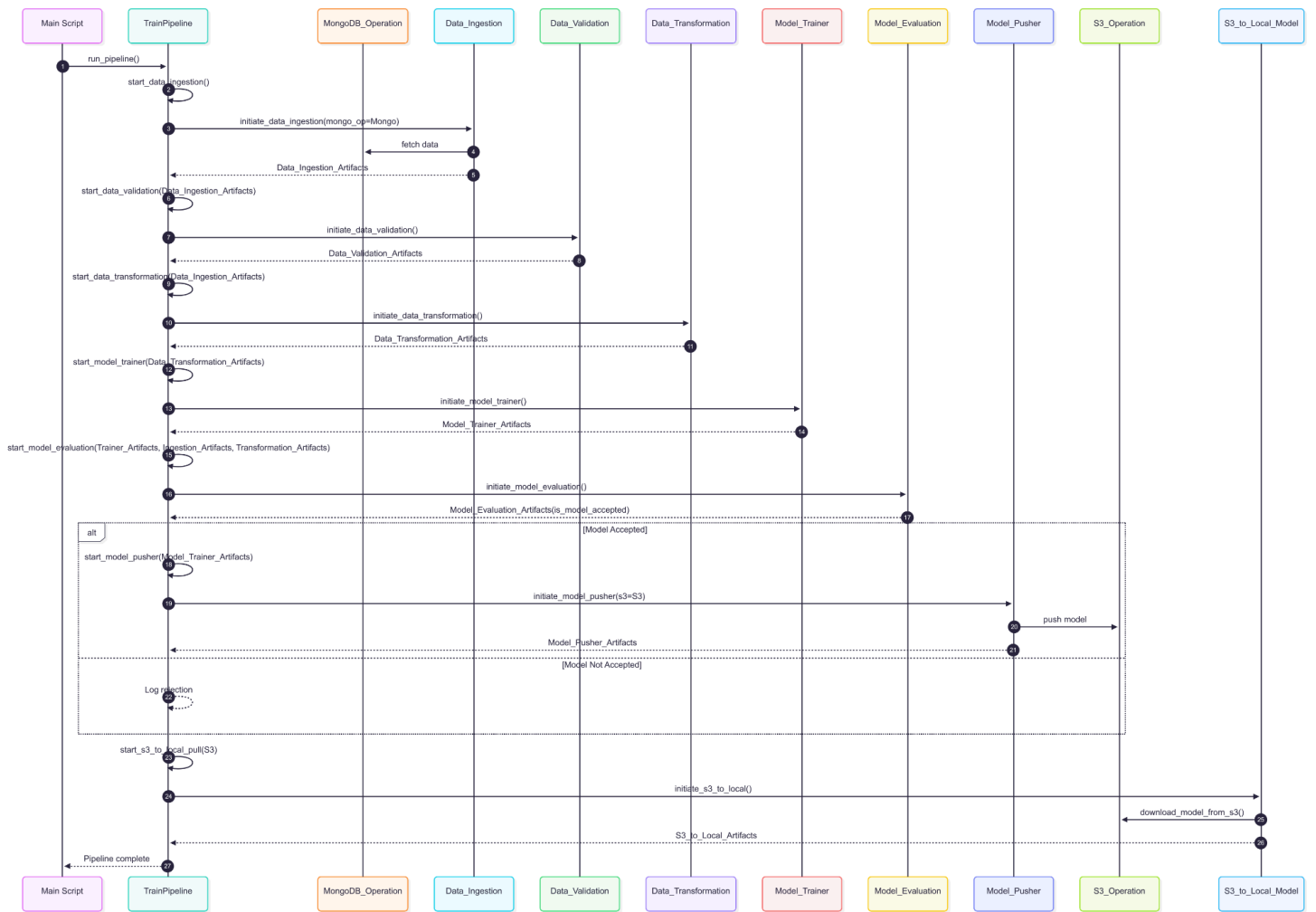
Explanation of Flowchart Training Pipeline:

- **Start Training Pipeline:** The entry point of the training pipeline process.
- **Initialize TrainPipeline:** Creating an instance of the TrainPipeline class.
- **Call start_data_ingestion():** Invoking the method to start data ingestion.
- **Create Data_Ingestion instance:** Creating an instance of the Data_Ingestion class.
- **Initiate data ingestion:** Executing the data ingestion process.
- **Return Data_Ingestion_Artifacts:** Returning the artifacts from data ingestion.
- **Call start_data_validation(data_ingestion_artifact):** Invoking the method to start data validation.
- **Create Data_Validation instance:** Creating an instance of the Data_Validation class.
- **Initiate data validation:** Executing the data validation process.
- **Return Data_Validation_Artifacts:** Returning the artifacts from data validation.
- **Call start_data_transformation(data_ingestion_artifact):** Invoking the method to start data transformation.
- **Create Data_Transformation instance:** Creating an instance of the Data_Transformation class.
- **Initiate data transformation:** Executing the data transformation process.
- **Return Data_Transformation_Artifacts:** Returning the artifacts from data transformation.

- **Call `start_model_trainer(data_transformation_artifact)`:** Invoking the method to start model training.
- **Create `Model_Trainer` instance:** Creating an instance of the `Model_Trainer` class.
- **Initiate model training:** Executing the model training process.
- **Return `Model_Trainer_Artifacts`:** Returning the artifacts from model training.
- **Call `start_model_evaluation(data_ingestion_artifact, data_transformation_artifact, model_trainer_artifact)`:** Invoking the method to start model evaluation.
- **Create `Model_Evaluation` instance:** Creating an instance of the `Model_Evaluation` class.
- **Initiate model evaluation:** Executing the model evaluation process.
- **Return `Model_Evaluation_Artifacts`:** Returning the artifacts from model evaluation.
- **Check if the model is accepted:** Evaluating whether the model is accepted based on predefined criteria.
- **Log model not accepted:** Logging the outcome if the model is not accepted.
- **Log model accepted:** Logging the outcome if the model is accepted.
- **Call `start_model_pusher(model_trainer_artifacts)`:** Invoking the method to start the model pusher.
- **Create `Model_Pusher` instance:** Creating an instance of the `Model_Pusher` class.
- **Initiate model pusher:** Executing the model pusher process.
- **Return `Model_Pusher_Artifacts`:** Returning the artifacts from the model pusher.

- **Call start_s3_to_local_pull(s3_operations):** Invoking the method to pull the best model from S3 to local.
- **Create S3_to_Local_Model instance:** Creating an instance of the S3_to_Local_Model class.
- **Initiate S3 to local pull:** Executing the S3 to local pull process.
- **Return S3_to_Local_Artifacts:** Returning the artifacts from the S3 to local pull.
- **End Training Pipeline:** The exit point of the training pipeline process.
- **Error Handling:** Handling any exceptions that occur during each method call.

Sequence Diagram here:



4.2 Prediction Pipeline

Description: Takes new shipment data → Prepares it → Loads best model → Predicts cost → Returns output.

Flowchart of Prediction Pipeline here:

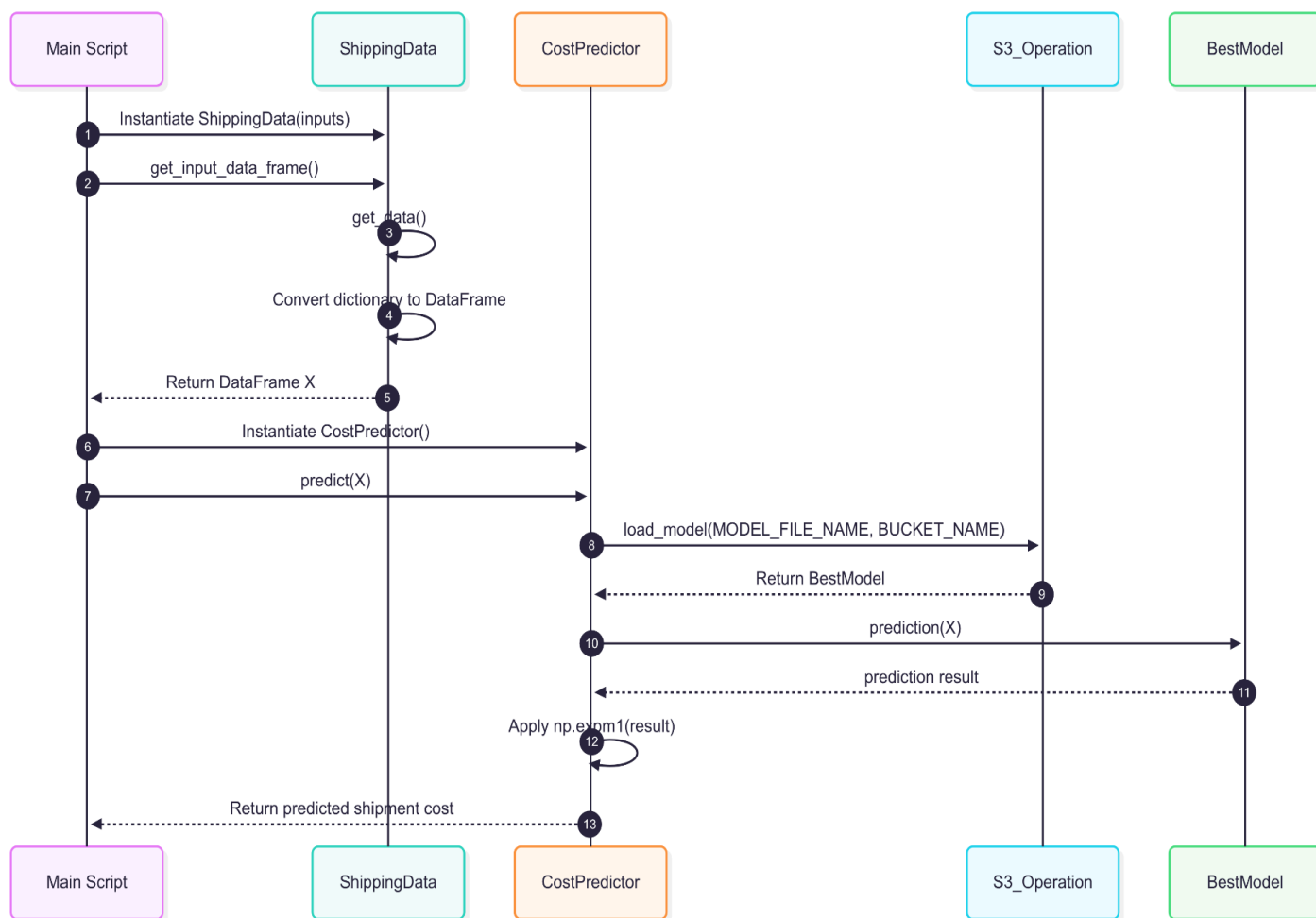


Explanation of Flowchart Prediction Pipeline :

- **Start Shipping Data Process:** The entry point of the shipping data process.
- **Initialize ShippingData:** Creating an instance of the ShippingData class with input parameters.
- **Call get_data():** Invoking the method to gather input data into a dictionary.
- **Create input_data dictionary:** Constructing the dictionary with input data.
- **Return input_data:** Returning the created input data dictionary.
- **Call get_input_data_frame():** Invoking the method to convert the input data dictionary into a DataFrame.
- **Call get_data():** Invoking the method again to gather input data.
- **Convert input_data to DataFrame:** Creating a DataFrame from the input data dictionary.
- **Return DataFrame:** Returning the DataFrame created from the input data.
- **End Shipping Data Process:** The exit point of the shipping data process.
- **Start Cost Prediction Process:** The entry point of the cost prediction process.
- **Initialize CostPredictor:** Creating an instance of the CostPredictor class.
- **Call predict(X):** Invoking the method to predict shipment costs using the best model.
- **The Load best model from S3:** Loading the best model from the S3 bucket.
- **Check columns of X:** Verifying the columns present in the input DataFrame .

- **Make predictions using best_model:** Using the loaded model to make predictions.
- **Apply expm1 to result:** Transforming the prediction result using the exponential function.
- **Check if the result is list or array-like:** Checking the format of the prediction result.
- **Return first element of result:** Returning the first element if the result is a list or array-like.
- **Check if the result is float:** Checking if the result is a single float value.
- **Return result:** Returning the result if it is a float.
- **Return None:** Returning None if the result is not in the expected format.
- **End Cost Prediction Process:** The exit point of the cost prediction process.
- **Error Handling:** Handling any exceptions that occur during the prediction process.
- **Raise CustomException:** Raising a custom exception if an error occurs.

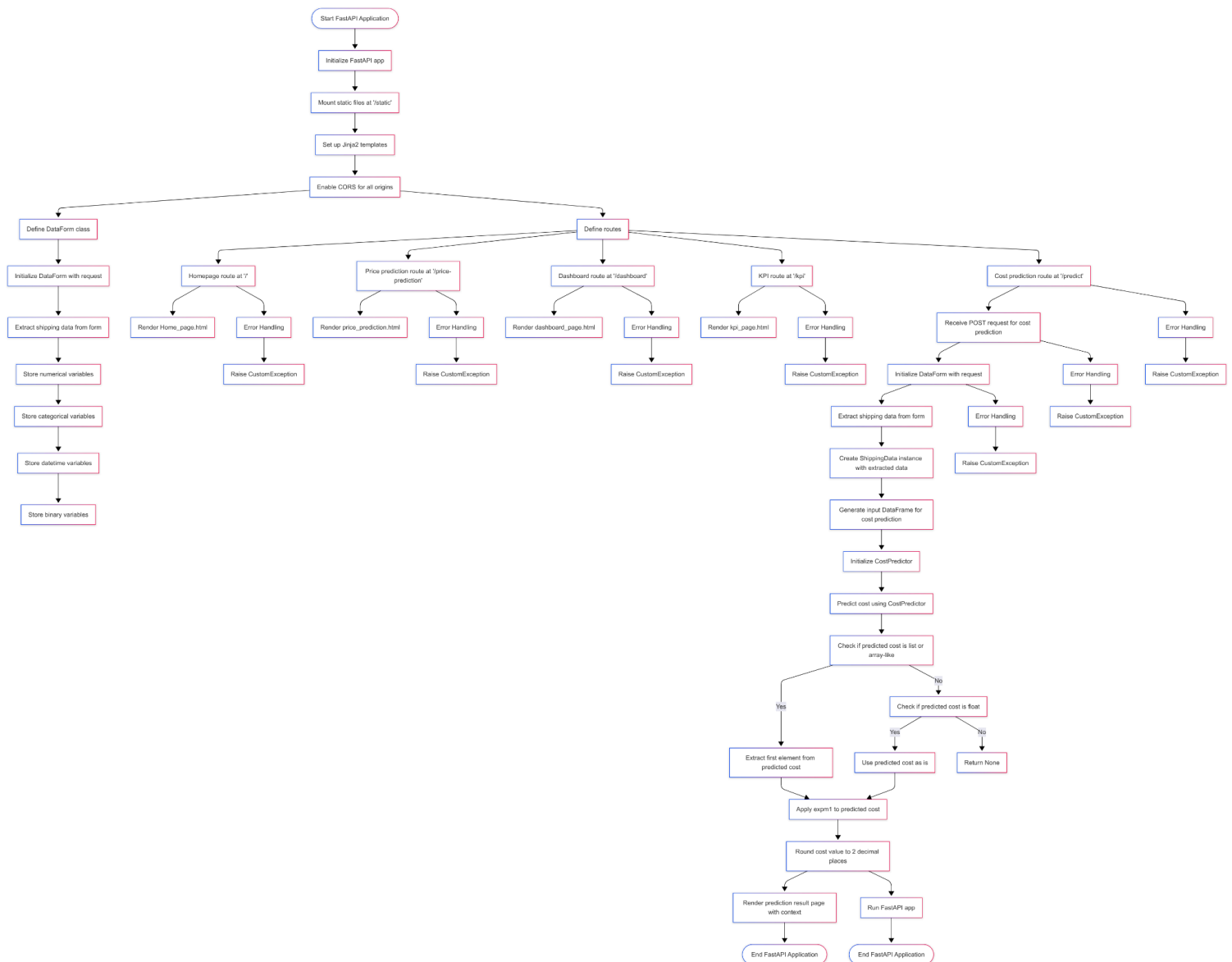
Sequence Diagram here:



5. Application Layer (FastAPI)

The **FastAPI app (app.py)** provides a web-based interface for price prediction, dashboard, and KPIs.

Flowchart of Application here:



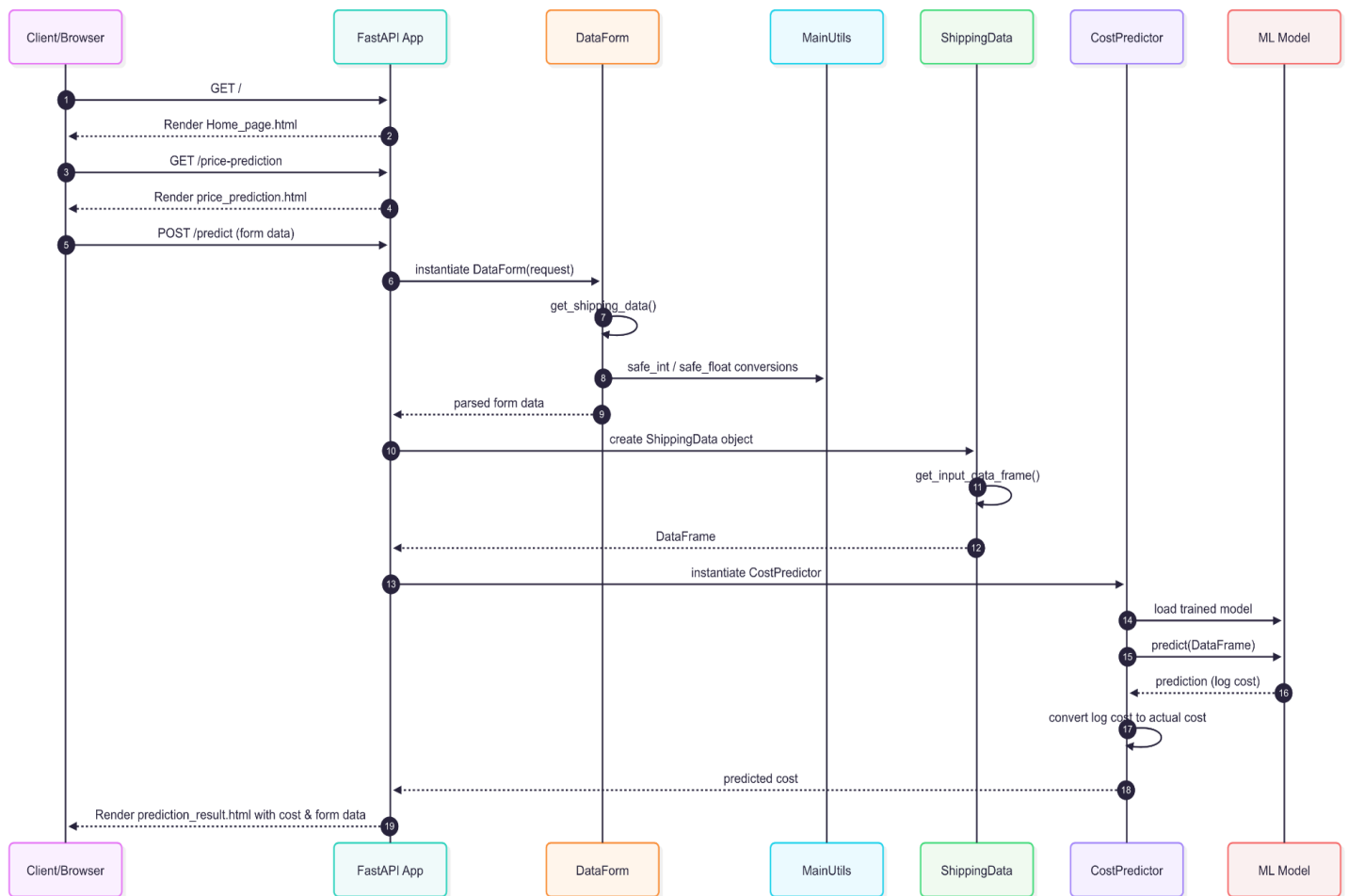
Explanation of Flowchart of Application :

- **Start FastAPI Application:** The entry point of the FastAPI application.
- **Initialize FastAPI app:** Creating an instance of the FastAPI application.
- **Mount static files at '/static':** Configuring the application to serve static files.
- **Set up Jinja2 templates:** Configure the application to use Jinja2 for HTML rendering.
- **Enable CORS for all origins:** Allowing cross-origin requests for testing purposes.
- **Define DataForm class:** Creating a class to manage user form submissions.

- **Initialize DataForm with request:** Creating an instance of the DataForm class with the incoming request.
- **Extract shipping data from form:** Extracting data from the submitted form.
- **Store numerical variables:** Storing numerical input values from the form.
- **Store categorical variables:** Storing categorical input values from the form.
- **Store datetime variables:** Storing date-related input values from the form.
- **Store binary variables:** Storing binary input values from the form.
- **Define routes:** Setting up the various API routes for the application.
- **Homepage route at '/':** Defining the route for the homepage.
- **Render Home_page.html:** Rendering the homepage template.
- **Price prediction route at '/price-prediction':** Defining the route for the price prediction form.
- **Render price_prediction.html:** Rendering the price prediction template.
- **Dashboard route at '/dashboard':** Defining the route for the dashboard.
- **Render dashboard_page.html:** Rendering the dashboard template.
- **KPI route at '/kpi':** Defining the route for the KPI page.
- **Render kpi_page.html:** Rendering the KPI template.
- **Cost prediction route at '/predict':** Defining the route for cost prediction.
- **Receive POST request for cost prediction:** Handling the incoming POST request for predictions.
- **Initialize DataForm with request:** Creating a new instance of DataForm to handle the request.
- **Extract shipping data from form:** Extracting the shipping data from the form submission.
- **Create ShippingData instance with extracted data:** Creating an instance of ShippingData with the extracted form data.
- **Generate input DataFrame for cost prediction:** Creating a DataFrame from the ShippingData instance for prediction.

- **Initialize CostPredictor:** Creating an instance of the CostPredictor class.
- **Predict cost using CostPredictor:** Using the CostPredictor to make a prediction based on the input DataFrame.
- **Check if the predicted cost is list or array-like:** Verifying the format of the predicted cost.
- **Extract first element from predicted cost:** Extracting the first element if the predicted cost is a list or array-like.
- **Check if the predicted cost is float:** Verifying if the predicted cost is a single float value.
- **Use predicted cost as is:** Using the predicted cost directly if it is a float.
- **Return None:** Handling cases where the predicted cost is not in the expected format.
- **Apply expm1 to predicted cost:** Transforming the predicted cost using the exponential function.
- **Round cost value to 2 decimal places:** Rounding the final cost value for display.
- **Render prediction result page with context:** Rendering the result page with the prediction and input data.
- **End FastAPI Application:** The exit point of the FastAPI application.
- **Error Handling:** Handling any exceptions that occur during various stages of the application.
- **Raise CustomException:** Raising a custom exception if an error occurs.

Sequence Diagram here:



6. Deployment & Infrastructure

Our system is designed to be **modular, reusable, and scalable**, with clear separation between **model training** and **model serving**. The current deployment runs in AWS cloud with local training support, and can be extended for future scalability.

6.1 Storage (AWS S3)

- Whenever we run the training pipeline (via **DVC**), the **best-performing model** is selected and pushed to **Amazon S3**.
- S3 is the **central storage hub** for:
 - Trained models (here, just pushed this)

- Versioned datasets
 - Model artifacts
 - Benefits:
 - Always have a **backup of production-ready models**.
 - Easy retrieval of the **latest model** for serving in EC2 containers.
 - Ensures models are consistent across training and serving.
-

6.2 Training Pipeline (Local / Cloud)

Current Setup (Local Training):

- Training is triggered **locally** using **DVC** ([dvc repro](#)).
- The pipeline automatically handles:
 - Data ingestion
 - Validation
 - Transformation
 - Model training and evaluation
- The **best model** from the run is pushed to **S3**.
- **MLflow** logs all experiment details:
 - Model parameters
 - Training and validation metrics
 - Model artifacts

Cloud Setup (What was done):

- After local experiments, the workflow was moved to the cloud for deployment:
 - An **EC2 instance** was launched (Ubuntu/Linux).
 - **Docker** was installed on EC2.
 - The **Docker image** was built **inside EC2**, not locally.
 - The built image was pushed to **Amazon ECR (Elastic Container Registry)**.
 - The same EC2 (or another one) **pulled the image from ECR** and ran the container.

Separation of Concerns:

- **Training:** Happens via **DVC pipelines** (local or could be extended to EC2/SageMaker).
 - **Serving:** Happens via **Dockerized FastAPI apps** on EC2.
 - The **FastAPI app does not retrain models**. It only loads the **best model from S3** (or bundled in Docker).
 - Retraining is triggered separately by running **dvc repro**, not by the API.
-

6.3 Model Serving (FastAPI)

- The deployed **FastAPI application (app.py)** is responsible for serving predictions.
- **Functionality:**
 - Loads the **latest best model** from **S3** (or from inside the container).
 - Provides REST endpoints for clients to submit shipment details and receive predicted prices.
- **Current deployment:**

- Runs on **HTTP** (functional but not secure).
 - **Future enhancement:**
 - Enable **HTTPS** using:
 - SSL/TLS certificates (e.g., Let's Encrypt, AWS Certificate Manager).
 - This will encrypt all communication between clients and server.
-

6.4 Containerization (Docker + AWS ECR + EC2)

- The entire application (code, dependencies, FastAPI server, model-loading logic) is **containerized with Docker**.
 - Deployment Workflow:
 - On EC2, **Docker is installed** and the **Docker image is built** from the code repository.
 - The image is pushed to **Amazon ECR**.
 - EC2 pulls the image back from **ECR** (ensuring version control of container builds).
 - When the container runs, it automatically **launches FastAPI** and serves predictions.
 - Security groups in AWS EC2 provide controlled external access via a **public HTTP endpoint**.
 - **Benefits of Dockerization:**
 - Consistent runtime across local and cloud environments.
 - Easy deployment and scaling (same image can run on multiple servers).
 - Isolated dependencies (no OS-level conflicts).
 - Reproducible deployments (ECR stores exact container versions).
-

6.5 Experiment Tracking (MLflow)

- **MLflow** is integrated into the training pipeline.
 - Every training run is logged with:
 - Model parameters
 - Metrics (training/validation accuracy, loss)
 - Artifacts (saved models)
 - This makes it easy to compare experiments and roll back to the **best model**.
-

6.6 Data Versioning (DVC)

- **DVC** (Data Version Control) manages:
 - Training datasets
 - Model outputs
 - Entire pipelines
 - Benefits:
 - Reproducibility: any past run can be re-created.
 - Collaboration: code and data are versioned just like Git.
 - Automatic handling of data and model dependencies.
-

7. Error Handling & Logging

- Every stage of the pipeline (ingestion, validation, transformation, training, serving) includes **error handling**.
 - If an error occurs:
 - A **CustomException** is raised.
 - Logs capture details of the error (timestamp, module, error message).
 - Logs are stored in **files** for monitoring and debugging.
 - The pipeline and API **fail gracefully** instead of crashing abruptly.
-

8. Security & Compliance

- **S3 credentials** are never hardcoded — they are managed via **environment variables** or **IAM roles**.
 - **Access to APIs** can be restricted to authorized users.
 - Current serving is on **HTTP**, but **HTTPS** will be implemented for secure, encrypted communication.
 - During training and prediction, **sensitive shipment/customer data** is handled securely to ensure compliance with data protection standards.
-

9. Conclusion

The **Shipment Price Prediction System** is an **end-to-end modular ML architecture** that separates training and serving for clarity, scalability, and maintainability.

- **Training:**

- Managed through **DVC pipelines** (currently local, can be extended to EC2/SageMaker).
- Best models are pushed to **S3** and tracked in **MLflow**.

- **Serving:**

- **Dockerized FastAPI app** deployed on EC2.
- **Docker image built in EC2**, pushed to **ECR**, and pulled for deployment.
- Current serving is via **HTTP**, with a roadmap to **HTTPS** for secure communication.

- **Governance & Reliability:**

- **MLflow** for experiment tracking.
- **DVC** for reproducibility and versioning.
- Central **logging and error handling** for easier debugging.

- **Future Enhancements:**

- Auto-scaling using AWS ECS or Kubernetes.
- Full cloud-based training pipelines.
- Advanced IAM-based fine-grained access control.